

**E DEVISER**

CONFIDENTIAL · INTERNAL ENGINEERING AUDIT

Edeviser Platform Full-Profile Audit & Remediation Plan

Every role · every flow · live-verified against production · with a candid
root-cause retrospective.

Scope Admin · Coordinator · Teacher · Student · Parent + shared infrastructure

Method Static code trace (file:line) cross-checked twice against the live database

Environment Production — **e-deviser.vercel.app** · Supabase `cdlgtbvxlxjpcddjazzx`

Generated June 2026 · **Mode** Audit only — no production code changed by this report



0 · How to read this document

This audit was produced in five passes plus a verification pass. **Pass 1** traced the React/TypeScript source and the Supabase edge functions for every role, citing `file:line`. **Pass 2** re-ran the material claims against the *live* production database (schemas, row counts, RLS, XP ledger). **Pass 3** (§2.5) authenticated as a real demo user in each role and exercised RLS-scoped reads, the leaderboard RPC, real write paths, and a cross-profile/cross-institution isolation matrix. **Pass 4** (§2.6) went further: for every *empty* feature domain it **seeded live data through the real flow, invoked the dependent edge function, and observed the actual result**. **Pass 5** (§2.7) tested the AI layer end-to-end with a live OpenRouter/Kimi key, fixed four AI defects, and re-verified them in production (real tutor chat, quiz generation). Finally, a dedicated **verification & cross-reference pass** (§11) re-checked every issue against current code + live schema, separated real bugs from seed artifacts, and mapped fix blast-radius. Where a later pass changed the verdict, the document says so plainly — guesses are not presented as facts.

The report is organised so a non-author can act on it:

- **Section 1 — Executive summary & scorecard:** the health of each role at a glance.
- **Section 2 — Live ground truth:** the numbers Pass 2 pulled from production, and which static claims they confirmed or corrected.
- **Sections 3–7 — Per-role findings:** feature-by-feature status tables + ranked issues (BLOCKER / HIGH / MEDIUM / LOW), each with evidence.
- **Section 8 — Cross-cutting infrastructure:** realtime, notifications, Sentry consent, RLS, i18n, CORS, dead code.
- **Section 9 — The badge-XP saga retrospective:** a candid answer to "why did this happen, why wasn't it caught, and why did the earlier approach fail?" — the questions you explicitly asked for.
- **Section 10 — Prioritised remediation plan:** what to fix, in what order, with effort and risk.
- **Section 11 — Verification & cross-reference appendix:** every issue re-checked against live code + live schema, real-bug vs seed-artifact, and a fix blast-radius / "don't-break-these" map.

Severity key. **BLOCKER** a core promised feature is non-functional in production · **HIGH** a feature is broken or silently wrong for real users · **MEDIUM** correctness/UX/security gap with a workaround · **LOW** hygiene, dead code, or type-safety debt.

1 • Executive summary & scorecard

The platform is **broad and largely functional** — auth, RLS isolation, OBE CRUD, the XP ledger, leaderboard, AI tutor, marketplace, challenges and most admin/coordinator/teacher management surfaces work end-to-end. The problems cluster in three places: **(a)** two PDF-generation edge functions that query a schema that no longer exists, **(b)** the student daily-engagement loop (streak, perfect-day, habit recording, journal/grade XP) which is wired in the UI but never actually fires the backend, and **(c)** a privacy-consent gap and a handful of RLS/CORS/realtime omissions in shared infrastructure.

Overall verdict: NOT yet release-clean. Confirmed BLOCKERs in accreditation output, student engagement, parent access, and privacy consent — now verified by acting as a real user of every role. None are data-loss risks and none breach multi-tenant isolation (which tested leak-free); all are fixable without schema redesign. The recently-deployed badge fix is verified working in production.

Update since first issue (fixes already deployed): the AI layer has been repaired and re-verified live — the **AI tutor now holds real Kimi-powered conversations** (with the academic-integrity guard working), **AI quiz generation produces real questions**, and the role-gate bug on `ai-feedback-draft` / `ai-module-suggestion` is fixed. See **§2.7**. Remaining AI gap: RAG course-material citations need an embeddings provider. The accreditation-PDF, planner, parent-RLS, engagement-XP and consent blockers below are still open.

Headline correction from live + seed-and-test: the accreditation PDF functions reject every coordinator/admin with a **403** before any query (role read from the JWT, which doesn't carry it). The same role-gate bug also breaks `ai-feedback-draft`. Driving the empty domains surfaced three more BLOCKERs: planner-task create/complete violates a DB CHECK (`pending` vs `todo`), AI quiz generation 404s on a non-existent `courses.institution_id` column, and study-session/wellness/planner/goal XP all 403 because the client awards XP as the student. The XP ledger, multiplier engine, marketplace, and multi-tenant isolation all tested healthy.

Role scorecard

Role	Working core	Blocker	High	Medium	Low	Headline issue
Admin	User/program/course/ILO CRUD, settings, audit log, bonus XP, invites, dashboards	2	1	1	2	Accreditation PDF blocked twice: 403 role gate + column drift (empty sections)
Coordinator	PLO mgmt, outcome mappings, CQI plans, curriculum matrix structure, dashboards	2	0	1	1	Course-file PDF blocked twice: 403 role gate + 4 column-drift queries
Teacher	Dashboard, assignment/CLO/rubric CRUD, attendance, quiz-gen, AI feedback, gradebook CSV	0	2	1	3	Grading-queue course filter ignored (non-linner join); level drift on grade XP
Student	Dashboard XP/level, courses, submissions, learning path, leaderboard, badges, tutor, marketplace, challenges	2	3	3	3	Streak never advances on login; Perfect-Day 50 XP never paid; journal/grade XP not awarded
Parent	Linked-child list, gamification view, planner (read-only), RLS verified-link enforcement	1	3	2	3	Progress + Attendance pages return empty (no parent RLS policy on <code>student_courses</code>)
Shared	RouteGuard, realtime manager, notifications read, RLS isolation	1	1	0	2	Sentry initialises before consent (FERPA/GDPR); notification realtime hook never mounted

The five things to fix first

- Accreditation report & course-file PDFs (Admin/Coordinator BLOCKERs — TWO stacked bugs each).** Live testing showed they first reject every coordinator/admin with a **403** (role read from the JWT, which doesn't carry it — it's in `profiles`), and even past that they query a pre-migration schema and emit empty sections. These are the artefacts an institution shows an accreditation body — currently fully unusable. `generate-accreditation-report`, `generate-course-file`
- Student engagement loop (Student BLOCKERs).** Login doesn't advance the streak; the 4th daily habit doesn't pay the 50 XP Perfect Day; journal and grade actions advertise XP they never grant. The gamification headline numbers are seed data, not live behaviour.
- Sentry consent gate (Shared BLOCKER).** `initSentry()` runs unconditionally at app load, bypassing the consent-gated path — error/replay capture starts without analytics consent.
- Parent data access (Parent BLOCKER).** Progress & Attendance read `student_courses` first, which has no parent RLS policy, so the pages are empty for every parent.
- CORS typo + notification realtime (HIGH).** `x-content-type` instead of `x-client-info` can break a browser-invoked function; the live-update bell hook is never mounted.

2 · Live ground truth (Pass 2) — what the database actually says

These figures were read directly from production and are the backbone of the audit. Several *overturned* or *refined* the static (Pass 1) conclusions — those are flagged.

PROFILES

124

71 student · 36 parent · 8 teacher · 6 coord · 3 admin

XP LEDGER

4,119

xp_transactions rows

BADGES

56

all scope=individual (fix live)

BADGE DEFS

68

seeded across 2 institutions

EVIDENCE

2,577

grade—evidence cascade alive

ATTAINMENT

1,743

course/program/student_course

HABIT_LOGS

0

Today View reads this → empty

BADGE XP TXNS

2

first ever — post-fix

Static claims that Pass 2 CONFIRMED

Claim (Pass 1)	Live evidence (Pass 2)	Verdict
outcome_mappings uses source/target, not child/parent	Columns: id, source_outcome_id, target_outcome_id, weight, created_at	CONFIRMED drift
submissions has no score_percent	Column count for score_percent = 0 (scores live on grades)	CONFIRMED drift
graduate_attributes has no title/code	Columns: id, institution_id, name, description, sort_order, created_at	CONFIRMED drift
cqi_action_plans uses action_description/program_id	No title/gap_description/corrective_actions/course_id columns exist	CONFIRMED drift
attainment scope enum has no PLO/ILO/CLO	Enum = student_course, course, program, institution only	CONFIRMED drift
Parent has no RLS policy on student_courses	parent policy count on student_courses = 0	CONFIRMED gap
Perfect-Day / journal XP not awarded at runtime	journal source = 0 rows; all perfect_day / login rows carry mid-May seed timestamps only	CONFIRMED not-wired
Today View table habit_logs is empty	habit_logs = 0 rows; habit_tracking = 3,010 (seed)	CONFIRMED split

Static claims that Pass 2 CORRECTED or REFINED — intellectual honesty

Earlier claim	Live reality	Refinement
"xp_total drifts from SUM(xp_transactions)"	0 students drift — every xp_total equals its ledger SUM	The value invariant holds today (award-xp recomputes the SUM). The residual risk is narrower: the grade DB-trigger path doesn't recompute level or apply multipliers. Downgraded from "broken invariant" to "level/multiplier drift risk."
"Streaks are dead / always zero"	Streaks span 0–31 (e.g. 5 students at 31, 2 at 7)	Those values are seed data. The accurate statement is "no runtime path advances streak_current on login," not "streaks are zero." The chip renders a frozen seed value.
"badges system fully dead (badges=0)"	56 individual badges now live; 2 badge-XP txns recorded	This was true before the recent fix. The fix is deployed and verified — see §9. The report treats badges as WORKING and documents the saga as a retrospective.
"badge_definitions empty"	68 rows across the 2 active institutions	Seeded during the fix. Spotlight rotation now has data to operate on.
"teams entirely absent (teams=0)"	1 team + members exist	Seeded for demo. Team-gamification dual-table hazard (below) remains latent but is no longer purely theoretical.

Why this matters: the previous audit doc (docs/audit-findings/03-gamification-flows.md) was written before live verification and before the badge fix. Treating it as current would have produced a wrong report. This is the core reason the user asked for the audit to be run twice.

2.5 · Live functional test (Pass 3) — acting as a real user of every role

Honest scope note. Passes 1–2 were a *code trace + database read*. Most per-role verdicts were therefore "the code path is wired / not wired" by inference, not "the action was performed and observed." This third pass closes that gap: **authenticate as a real demo user in each of the five roles against production** and drive the actual backend the way the UI does — RLS-scoped reads, the leaderboard RPC, real `award-xp` / `check-badges` / `process-streak` calls, the two PDF edge functions, plus a deliberate cross-profile and cross-institution isolation matrix. **47 live checks** were executed. Results below confirmed, corrected, or refined the static findings, and surfaced one new BLOCKER.

3a · What each role could actually do (real authenticated sessions)

Role / account	Live actions & reads	Observed result	Verdict
Student · student01 (Gulf)	own gamification / courses / badges / assignments; leaderboard RPC; <code>award-xp(submission)</code> with a real enrolled assignment; <code>check-badges</code> self	XP=1870 L11; courses=3; badges=3; assignments=12; leaderboard=5 rows; <code>award-xp</code> ✓ (+13 XP, 15×0.9 level mult); <code>check-badges</code> 200 ok	Core loop works
Teacher · anderson (Gulf)	own courses, submissions, attendance, CLOs, grades; <code>send_teacher_nudge</code> RPC	courses=1, submissions=100, attendance=900, CLOs=9, grades=100; <code>nudge</code> RPC ✓	Reads + nudge work
Coordinator · curriculum (Gulf)	PLOs, outcome_mappings, CQI, attainment, graduate_attributes; <code>generate-course-file</code>	PLOs=3, mappings=18, CQI=0, attainment=630, GA=2; <code>course-file</code> → 403 Forbidden	Course-file blocked
Admin · principal (Gulf)	users, audit_logs, xp_events, badge_definitions; <code>generate-accreditation-report</code>	users=51, audit=5, xp_events=0, badge_defs=34; <code>accreditation</code> → 403 Forbidden	Report blocked
Parent · parent01 (Gulf)	verified links + every child-data table the pages touch	links=1; <code>student_courses</code> =0; attainment=21; attendance=90; gamification=1; courses=3	Confirms + refines P-1
process-streak (student02)	called <code>process-streak(student_id,type:'login')</code> directly	200 ok → <code>streak_count:1</code> , <code>last_login</code> set to today	Function works — login just never calls it

3b · Cross-profile & cross-institution data scoping (the multi-tenant promise)

The platform's core design is that each user gets only their own role- and institution-scoped data. I tested this by deliberately attempting to read and write across those boundaries. **Every isolation check passed.**

Boundary test (performed live)	Expectation	Result
Gulf student reads raw peer <code>student_gamification</code> rows	blocked (peers only via leaderboard RPC)	0 rows — PASS
Gulf student reads Noor (other-institution) <code>profiles</code>	blocked	0 rows — PASS
Gulf teacher reads Noor students	blocked	0 rows — PASS
Gulf admin reads Noor users	blocked (institution-scoped)	0 of 51 are Noor — PASS
Noor admin reads Gulf users	blocked	sees own 68, Gulf=0 — PASS
Parent reads an <i>unlinked</i> child's gamification	blocked (verified link only)	0 rows — PASS
Student calls <code>award-xp</code> targeting ANOTHER student	403 Forbidden	403 — PASS (ownership enforced in-handler)
Student calls <code>check-badges</code> for ANOTHER student	403 Forbidden	403 — PASS
Per-institution data actually differs	distinct datasets	Gulf avg XP 1145 vs Noor 1293 — PASS

Verdict on multi-tenancy: the RLS isolation and the edge-function ownership checks are **solid**. No cross-role or cross-institution leak was found in 9 deliberate boundary probes, and malicious cross-user writes are rejected with 403. This is the strongest part of the platform.

3c · New / corrected findings that ONLY the live test surfaced

#	Finding	How the live test changed it
L-1	NEW BLOCKER — both PDF edge functions reject every valid coordinator/admin with 403 , before any query runs.	Live, a genuine coordinator (<code>role='coordinator'</code> in <code>profiles</code>) calling <code>generate-course-file</code> and a genuine admin calling <code>generate-accreditation-report</code> both got 403 "admin or coordinator role required" . Root cause, confirmed by decoding the live JWT: the functions read the role from <code>user_app_metadata?.role</code> / <code>user_metadata?.role</code> (<code>generate-course-file/index.ts:440-442</code> , <code>generate-accreditation-report/index.ts:475-481</code>), but the JWT carries only <code>{provider, providers}</code> — the role lives in the <code>profiles</code> table, not the token. So the gate is empty-string for everyone and blocks 100% of legitimate callers. This sits <i>in front of</i> the column-drift BLOCKERS (A-1/C-1): the PDFs never even reach the bad queries today. Both bugs must be fixed for the feature to work.
L-2	Leaderboard RPC is FINE — a false alarm caught and dismissed.	A first probe failed ("function not found"); on inspection the live signature is (<code>p_institution_id</code> , <code>p_limit</code> , <code>p_offset</code>) and the real client (<code>useLeaderboard.ts:195-198</code>) calls it correctly — re-ran with the right args and got 5 rows. Not a product bug; documented for transparency.
L-3	Parent P-1 mechanism pinned precisely (still a real BLOCKER) .	A parent sees <code>student_courses=0</code> but CAN see <code>outcome_attainment=21</code> , <code>attendance=90</code> , <code>courses=3</code> . So data isn't inaccessible — the page <i>short-circuits</i> : <code>ParentProgressPage.tsx:47-55</code> and <code>ParentAttendancePage.tsx:44-50</code> read <code>student_courses</code> first and <code>return []</code> when it's empty. Fix options: add a parent SELECT policy on <code>student_courses</code> (also missing on <code>course_sections</code> , <code>class_sessions</code>), OR rewrite the page to derive courses from the already-visible attainment path.
L-4	XP engine + badge chain verified working by real action .	<code>award-xp(submission)</code> with a real assignment returned 200 and correctly applied the level multiplier (15→13 at L11). <code>check-badges</code> self returned 200. <code>process-streak</code> per-student advanced the streak to 1. The gamification backend is healthy — the gaps are purely that <i>login/grade/journal/perfect-day client flows never call these functions</i> .
L-5	Streak/journal gaps confirmed behaviourally (were inference).	<code>process-streak</code> works when called, yet login leaves <code>last_login_date</code> stale and writes no <code>habit_tracking</code> row; a real journal create produced 0 <code>journal</code> XP rows. Confirms S-1 and S-5 by observation, not just code reading.
L-6	No test pollution .	The deliberate bad-write probe (fake assignment id) was correctly rejected with 400 and wrote 0 rows; the cross-user write attempts were 403. Verified the ledger gained no stray <code>e2e-*</code> rows afterwards.

Net effect of Pass 3. It *confirmed* the four engagement/parent BLOCKERs by real action, *verified* the multi-tenant isolation is leak-free across 9 boundary probes, *corrected* one false alarm (leaderboard), and *surfaced one new BLOCKER* (PDF role-gate 403) that a pure code/schema read had missed because it only manifests with a real JWT. The two PDF functions are now known to have **two stacked blockers each**: the 403 role gate (L-1) in front of the column drift (A-1/C-1).

2.6 · Strict seed-and-test (Pass 4) — populate every empty domain, then run it

Scope. The remaining inference gap was features with *no seeded data*. This pass closes it the only honest way: as the real demo users, **create the missing data through the actual tables/flows, invoke the dependent edge function, and observe**. I first seeded the foundational `institution_settings` (it was empty for all institutions — several functions read it) and kept it. All other probe rows were created, observed, and then deleted (verified clean; `xp_transactions` is append-only by design so probe XP used real idempotent sources). This pass surfaced **three new BLOCKERS** and a systemic XP-authorization defect that only a real write could reveal.

4a · New BLOCKERs found only by performing the write

#	Feature	What happened live	Root cause
N-1	Planner tasks — create & complete both broken	Inserting a task as the real student with the app's value <code>status:'pending'</code> → violates <code>check constraint planner_tasks_status_check</code> . Re-running with <code>status:'todo'</code> + update <code>'done'</code> succeeded.	The DB CHECK allows only <code>todo/in_progress/done/deferred</code> , but <code>usePlannerTasks.ts:59</code> writes <code>'pending'</code> and <code>:197</code> writes <code>'completed'</code> (hidden by an <code>as never</code> cast). Every planner create and completion fails in production. BLOCKER
N-2	AI quiz generation — dead for every course	Calling <code>generate-quiz-questions</code> as the course's real teacher returned <code>404 "Course not found"</code> for a course that demonstrably exists.	<code>generate-quiz-questions/index.ts:571</code> selects <code>courses.institution_id</code> , but <code>courses</code> has no <code>institution_id</code> column (institution is via <code>program_id=programs</code>). The bad select returns null → false 404. Column-drift class, confirmed against live schema. BLOCKER
N-3	Non-core XP sources can never be awarded from the client	Completing a real study session, logging a wellness habit, completing a planner task, or meeting a weekly goal all call <code>award-xp</code> with the student's own JWT and received 403 Forbidden (silently swallowed by fire-and-forget try/catch).	<code>award-xp/index.ts:605</code> only allows self-award for <code>login/submission/journal</code> ; every other source requires service-role. But <code>useSessionCompletion</code> , <code>useWellnessHabits</code> , <code>usePlannerTasks</code> , <code>useWeeklyGoalXP</code> all invoke it as the student. So study-session, wellness, planner-task, weekly-goal, and review XP are all dead . BLOCKER
N-4	LLM features: OpenRouter works; tutor blocked by an OpenAI dependency	After setting <code>OPENROUTER_API_KEY</code> (a free Kimi key) + <code>TUTOR_PRIMARY_MODEL=moonshotai/kimi-k2</code> in prod: <code>generate-plan-update</code> returned 200 with a real LLM study plan generated by Kimi ("Maintain 1 hour/week for light review and spaced repetition...") — the first successful LLM round-trip in the platform's history (log tables were empty). But <code>chat-with-tutor</code> still <code>500</code> s: its <i>first</i> step is an OpenAI embedding (<code>text-embedding-ada-002</code>) for RAG, which an OpenRouter key cannot serve.	Two distinct gaps: (1) <code>config</code> — <code>OPENROUTER_API_KEY</code> was unset (now set, verified working with Kimi); (2) hard OpenAI dependency — <code>chat-with-tutor</code> and <code>embed-course-material</code> call <code>api.openai.com/v1/embeddings</code> directly and have no OpenRouter/local fallback, so the AI tutor cannot run on an OpenRouter-only setup. Either add an <code>OPENAI_API_KEY</code> or route embeddings through an OpenRouter-compatible embedding model. HIGH
N-5	<code>generate-plan-update</code> — NOT NULL insert bug	With the LLM working, the function still <code>500</code> d on save: <code>null value in column "interaction_count"</code> violates <code>not-null constraint</code> when the caller omits <code>recent_interaction_count</code> . Supplying it → 200.	<code>tutor_plan_updates.interaction_count</code> is <code>integer NOT NULL</code> with no default; <code>generate-plan-update/index.ts:298</code> sets it from the request's <code>recent_interaction_count</code> but never validates/defaults it. The live caller (<code>chat-with-tutor</code>) is down, so this was never hit. Add validation or a default. MEDIUM

Live LLM verification (with user-supplied free Kimi key). The OpenRouter integration is **confirmed working end-to-end** — `generate-plan-update` produced a genuine Kimi-generated plan in production. The key is a free-tier OpenRouter account (limited credits: a probe with the default 32k `max_tokens` returned `402 Payment Required`; smaller requests succeed), which is fine for QA but would need a paid plan or per-function `max_tokens` caps for real usage.

2.7 · AI suite — fixed and re-verified live (Pass 5)

Outcome: the AI tutor and most AI functions now work on the free Kimi key. After live testing the AI layer, four code fixes were made and deployed, then re-verified by real calls. The tutor holds a genuine streamed conversation, and quiz generation produces real questions — both via `moonshotai/kimi-k2`. `tutor_llm_logs` now records **5 successful LLM calls** (was 0 — no AI call had ever succeeded before).

Fix	Problem found live	Change made & deployed	Re-verified result
F-1 · Tutor optional embeddings	<code>chat-with-tutor</code> hard-failed 500 when <code>OPENAI_API_KEY</code> absent (its first step was an OpenAI embedding for RAG; OpenRouter/Kimi has no embeddings API)	Made the embedding + pgvector retrieval optional — if no OpenAI key (or it fails), skip RAG and answer from persona + CLO context + history. <code>chat-with-tutor/index.ts:892-960</code>	WORKS. Real Socratic reply streamed: "Start by reviewing whole-number operations... with positive and negative integers." Integrity guard also fired: refused "write my homework" and redirected pedagogically. Emits a <code>handoff_suggestion</code> noting no materials found (graceful).
F-2 · Shared role-gate	<code>ai-feedback-draft</code> & <code>ai-module-suggestion</code> returned 403 to real teachers (read role from empty JWT <code>app_metadata</code>)	Resolve role from the <code>profiles</code> table by <code>user.id</code> (same root cause as the PDF functions). Both functions updated & deployed.	WORKS. <code>ai-module-suggestion</code> → 200 ("No weak CLOs found"); <code>ai-feedback-draft</code> passes the role gate (now validates payload).
F-3 · Quiz institution_id drift	<code>generate-quiz-questions</code> → 404 "Course not found" because it selected non-existent <code>courses.institution_id</code>	Derive institution via <code>program_id + programs(institution_id)</code> join; also read <code>TUTOR_PRIMARY_MODEL</code> and cap <code>max_tokens</code> at 4000. Deployed.	WORKS. Reached the LLM (404 gone).
F-4 · Kimi structured-output	Quiz then 502'd — Kimi rejected <code>response_format: {type:'json_object'}</code> . ("model does not support feature: structured-outputs")	Only send <code>response_format</code> for OpenAI models; otherwise rely on the prompt's "valid JSON" instruction + existing parse/validate. Deployed.	WORKS. <code>generate-quiz-questions</code> → 200 with real questions ("Which of the following is defined as the set of all possible output values of a function...").

Still requires attention. (a) **RAG is off** until an embeddings provider is configured — the tutor answers from persona/CLO context but cannot cite course materials (needs an `OPENAI_API_KEY` or an OpenRouter-compatible embedding model; `embed-course-material` remains OpenAI-only). (b) **generate-plan-update N-5 bug** (NOT NULL `interaction_count`) is still open — fix R-1f. (c) The free Kimi key has limited credits and **per-request token caps** are advisable. (d) `quiz_generation_logs` records the model label as the hardcoded constant rather than the actual model used — cosmetic logging inaccuracy.

4b · Domains seeded & exercised — what actually works

Domain (was empty)	Live action performed	Result
Study sessions	create + complete as student	Row CRUD WORKS; XP award 403 (N-3)
Weekly goals	create as student	WORKS
Wellness habits	log as student	Log WORKS; XP award 403 (N-3)
Discussions	create thread + reply as student	WORKS
Journal	create entry as student	Row WORKS; advertised +20 XP never awarded
Marketplace	<code>process-purchase</code> "AI Tutor Hint Pack" (30 XP)	WORKS — balance debited 1683→1653, purchase row created
Bonus XP event	admin creates 2× <code>bonus_weekend</code> event, student earns XP	WORKS — 20 base → 36 (2× × 0.9 level mult) verified

Streak driver	<code>process-streak</code> per-student	Function WORKS (streak→1); login flow just never calls it (S-1)
Habit correlations	<code>compute-habit-correlations</code>	Runs; returns <code>insufficient_data</code> (<code>habit_logs</code> empty)
Challenges	student attempts to join a challenge	RLS blocks INSERT — no student self-join policy on <code>challenge_participants</code> (only teacher-manage + read). Challenges are not student-joinable.
AI feedback draft	<code>ai-feedback-draft</code> as teacher	403 — same JWT role-gate bug as PDFs (reads role from token, not <code>profiles</code>)

The role-gate bug is wider than the two PDF functions. Live testing shows `ai-feedback-draft` (and the same pattern in `generate-course-file` , `generate-accreditation-report`) all read the caller role from the JWT's `app_metadata` / `user_metadata` , which carries only `{provider, providers}` — the role lives in `profiles` . **Every role-gated edge function using this pattern rejects 100% of legitimate callers with 403.** One shared fix (resolve role from `profiles` by `user.id`) repairs all of them.

Net effect of Pass 4. Seeding + driving the empty domains found **3 new BLOCKERs** (planner status mismatch N-1, quiz `institution_id` drift N-2, client XP-source 403 N-3) and a production secret gap (N-4) — none of which a code/schema read alone would have proven. It also *verified working*: marketplace purchase, bonus-XP multiplier, study/goal/wellness/discussion CRUD, and the streak function. The XP *ledger and multiplier engine* are healthy; the breakage is consistently at the *authorization boundary and the column/enum drift*, not the business logic.

3 · Admin role

Feature	Stack path	Status	Notes
User management (CRUD + soft-delete)	useUsers.ts:30-243 → profiles	WORKING	Audit-logged; <code>as never</code> casts at :48;139
Programs / Courses CRUD	usePrograms.ts, useCourses.ts	WORKING	Paginated, FK joins, audit-logged
ILO management	useILOs.ts:18-260 → learning_outcomes	WORKING	Delete checks mapping deps
Institution settings / accreditations	useInstitutionSettings.ts	WORKING	Columns verified in types
Audit log viewer	useAuditLogs.ts:38-78 → audit_logs	WORKING	Reader/writer columns aligned
Bonus XP events	useBonusEvents.ts → xp_events	WORKING*	Data path OK; audit diff under-records (MEDIUM)
Onboarding / invitations	useAdminDashboard.ts, send-invitation-email	WORKING	programId filter is a no-op (LOW)
Accreditation reports	useAccreditationReport.ts → generate-accreditation-report	BROKEN	Column drift → empty attainment/GA sections (BLOCKER)
Admin dashboard KPIs	useAdminDashboard.ts:24-110	WORKING	Parallel head-count queries

Ranked issues

- BLOCKER A-0 · 403 role gate blocks every admin (verified live).** `generate-accreditation-report/index.ts:475-481` reads the caller role from `user.app_metadata?.role ?? user.user_metadata?.role` . Decoding a real admin JWT shows it carries only `{provider, providers}` — the role lives in the `profiles` table. So the gate evaluates to empty and returns `403 "admin or coordinator role required"` for a genuine admin (observed live). The report is unreachable before the column-drift code even runs. Fix: resolve role/institution from `profiles` by `user.id` .
- BLOCKER A-1 · `generate-accreditation-report` column drift.** The function selects `outcome_attainment.score_percent` (real column is `attainment_percent`) and filters `scope === 'PLO' / 'ILO'` (the live enum only has `student_course/course/program/institution`), and reads `graduate_attributes.title/code` (real columns are `name/description`). All confirmed against live schema. Because the function only destructures `{ data }` with no `error` check, the bad query returns null and the PDF renders attainment as 0 / "Not Yet" and the Graduate Attribute section empty — with HTTP 200. Evidence: `generate-accreditation-report/index.ts:561-562,573-599,653-657` .
- A-2 · Accreditation report semester filter is dead.** `ReportGeneratorPage.tsx:30-33` declares a local `useSemesters` stub returning `[]` that shadows the real hook, so the semester dropdown is permanently empty.
- A-3 · Bonus-event audit logging records almost nothing.** `auditLogger.ts:31` allow-lists `["name","multiplier","start_date","end_date"]` but the table columns are `xp_multiplier/starts_at/ends_at` , so the audit diff only e
- A-4 · `as never` casts at `useUsers.ts:48,139` , `useCourses.ts:120` bypass generated Insert types.** **A-5 · pending-onboarding program filter ignored (`useAdminDashboard.ts:158-170`).**

4 · Coordinator role

Feature	Stack path	Status	Notes
PLO management	usePLOs.ts:30-300 → learning_outcomes	WORKING	Drag-reorder, dep-checked delete
Outcome mappings PLO→ILO	usePLOMappings → outcome_mappings	WORKING	Correct source/target columns
Curriculum matrix	useCurriculumMatrix.ts	WORKING / NO-DATA	Cell status is CLO-count placeholder, ignores attainment (:30-34)
CQI action plans	useCQIPlans.ts → cqi_action_plans	WORKING	App hook columns match table exactly
Course-file generation	useCourseFile.ts → generate-course-file	BROKEN	4 queries on non-existent columns (BLOCKER)
Coordinator dashboard KPIs	useCoordinatorDashboard.ts:14-110	WORKING*	"At-Risk" = active-student count placeholder

Ranked issues

- BLOCKER C-0 · 403 role gate blocks every coordinator (verified live).** `generate-course-file/index.ts:440-442` checks `user.app_metadata?.role` , which is empty on the real JWT — a genuine coordinator calling the function received `403 "admin or coordinator role required"` in live testing. Same root cause and fix as A-0 (read role from `profiles`).
- BLOCKER C-1 · `generate-course-file` — four broken queries.** Confirmed against live schema: (1) `outcome_mappings` selected with `child_outcome_id/parent_outcome_id` (real: source/target); (2) `submissions.score_percent` (does not exist — scores are on `grades`); (3) `outcome_attainment.score_percent` + filter `scope='CLO'` (column is `attainment_percent` ; enum has no CLO); (4) `cqi_action_plans` selected with `title/gap_description/corrective_actions` and filtered by `course_id` (none exist; table is keyed by `program_id` with `action_description/root_cause`). The course file is the accreditation artefact per course/semester — it ships with empty CLO-PLO mapping, sample-work, CLO-attainment and CQI sections. Evidence: `generate-course-file/index.ts:521-534,573-585,610-621,654-665` .
- C-2 · Curriculum matrix / dashboard show placeholder metrics, not real attainment (`useCurriculumMatrix.ts:30-34` , `useCoordinatorDashboard.ts:62-68`) — could mislead an accreditation reviewer.**
- C-3 · `as never` insert cast at `usePLOs.ts:96` .**
- Root cause shared by A-1 and C-1.** The application hooks consistently use the *current* schema; only these two Deno PDF functions use an *older* vocabulary. The schema was migrated (mappings renamed to source/target, attainment column renamed, scores moved to `grades` , CQI table redesigned) **without updating the two edge functions**, and because they never check the PostgREST `error` object, the regression shipped silently. Any fix must also add `if (error) throw` guards so the next drift fails loudly.

5 • Teacher role

Feature	Stack path	Status	Notes
Dashboard KPIs, course selector	useTeacherDashboard.ts	WORKING	Uses <code>assignments!inner</code> correctly
Assignment / CLO / rubric CRUD	useAssignments.ts, useCLOs.ts	WORKING	Bloom's levels present; stale casts (LOW)
Attendance entry	attendance hooks → attendance_records	WORKING	7,500 rows live
Quiz generation / AI feedback	generate-quiz-questions, ai-feedback-draft	WORKING	JWT-guarded; ai_feedback logged
Grade submission → evidence → attainment	useCreateGrade → DB trigger trigger_attainment_rollup	WORKING*	Runs via SQL trigger, not the edge fn; 2,577 evidence rows confirm
Grading queue course/assignment filter	useSubmissions.ts:88-90,138-141	BROKEN	Non-linner embedded filter → course ignored (HIGH)
Course materials embedding	embed-course-material edge fn	NOT-WIRED	No caller anywhere → RAG store never populated
Gradebook export	client-side CSV (bulk-grade-export fn missing)	WORKING	Hook references a fn dir that doesn't exist; UI uses client CSV

Ranked issues

- T-1 • Grading-queue filter ignored. `useSubmissions.ts` filters on a non-`!inner` embedded join (`assignments.course_id`), so PostgREST does not restrict parent rows — the selected course/assignment is effectively ignored and the que
- T-2 • Grade XP path skips level + multipliers. Grade release awards a flat +15 via direct `UPDATE student_gamification SET xp_total = xp_total + 15` in the DB trigger (idempotent ledger insert), but never recomputes `level` and n
- T-3 • Grading stats overcount. `useGradingStats` counts `submissions.status='submitted'` as "pending" but nothing ever sets `status='graded'` so the pending count overstates and disagrees with the KPI card.
- T-4 • embed-course-material orphaned (uploaded materials never embedded). T-5 • stale `as any` / `as never` in `useAssignments/useCLOs` with outdated "types not regenerated" comments though the columns now exist. T-6 • silent error-

Note on the calculate-attainment-rollup edge function. It is *orphaned* — the live grade→evidence→attainment cascade runs entirely through the SQL trigger `trigger_attainment_rollup`. If the edge function were ever wired in, it would violate `evidence.plo_id/iilo_id NOT NULL`. Leave it disconnected or delete it; do not "re-enable" it without schema review.

6 · Student role (largest surface)

Feature	Stack path	Status	Notes
Dashboard XP / level chips	useStudentDashboard.ts:40-95	WORKING	Advances via award-xp on submission
Dashboard streak chip	useStreak.ts:24; useStudentDashboard.ts:62-66	BROKEN	streak_current never incremented at runtime
Courses / assignments / submission	useSubmissions, AssignmentDetailPage:223-247	WORKING	Submission fires award-xp → badge chain
Learning path (Bloom order + prereq gate)	useLearningPath.ts:63-160	WORKING	Bloom from clo_weights[0] only (LOW)
Leaderboard + anonymity opt-out	get_leaderboard_page RPC	WORKING	Opt-out excluded server-side
Badges (BadgeCollection / useTieredBadges)	useTieredBadges.ts:44-49	WORKING*	Fix live; missing explicit scope='individual' filter
Habit heatmap	useHeatmapData.ts:68-71 (habit_tracking)	NO-DATA	Only read_content + seed populate it
Daily habits / Today view	useTodayView.ts:218 (habit_logs)	NO-DATA	habit_logs = 0 rows live
Journal	useJournal.ts:104-130	NOT-WIRED (XP)	Entry persists; "+20 XP" never granted (journal source = 0 live)
AI tutor (chat + tutorApi)	useTutorMessages, tutorApi	WORKING	SSE streaming, strong typing
Study sessions / planner / wellness / marketplace / challenges	various	WORKING	Planner/wellness correctly fire award-xp + check-badges

Ranked issues

- BLOCKER** **S-1 · Login never advances the streak.** `AuthProvider.signIn` only calls `logActivity({event_type:'login'})` (`AuthProvider.tsx:230-234`) which inserts into `student_activity_log` and nothing else — no `process-streak` invoke, no `award-xp('login')`, no `habit_tracking.login` write. The only `process-streak` caller is a midnight cron posting `{type:'midnight_reset'}`, but the function requires `student_id` and has no batch branch, so it 400s and processes nobody. Result: `streak_current`, streak badges and milestone XP are all driven by seed data only. (Pass 2: live streaks 0–31 are all mid-May seed values; `login` XP rows are seed-timestamped.)
- BLOCKER** **S-2 · Perfect Day 50 XP never paid.** `perfect_day` is a valid XP source and in `MILESTONE_SOURCES`, but nothing ever calls `award-xp({source:'perfect_day'})`. The 6 PM `perfect-day-prompt` cron only inserts a nudge notification that *advertises* the 50 XP (`perfect-day-prompt/index.ts:207-213`). `isPerfectDay()` is used only in tests. (Pass 2: the 565 `perfect_day` rows are seed data.)
- S-3 · Academic habit completions never recorded.** `login/submit/journal` are never written to `habit_tracking` or `habit_logs` by their real flows (only `read_content` via the read timer, and sessions via `session-completion`). This is the
- S-4 · Grade release awards no client XP / badge check.** `useCreateGrade` (`useGrades.ts:77-105`) inserts the grade and relies on the DB trigger for attainment, but never invokes `award-xp('grade')`; the grade—15XP—`perfect_score`
- S-5 · Journal XP not awarded despite "+20 XP" UI copy** (`useJournal.ts:104-130` , `StudentJournalPage.tsx:128,254`).
- S-6 ·** `useStudentBadges` missing `scope='individual'` filter and is dead code (imported nowhere); the live `useTieredBadges` filters `team_id IS NULL` instead. Works today, not contract-correct. **S-7 ·** `habit_logs` vs `habit_tracking` dup
- S-9 ·** pervasive `as never` / `as any` casts (teams/tiered-badges/xp-balance). **S-10 ·** learning path Bloom from first CLO only. **S-11 ·** fragile double-`.data` cast in `useStudentDashboard`.

Deep-dive: is `habit_logs` working with live data? (extensive test)

Verdict: NO. `habit_logs` is completely empty in production (0 rows) and is effectively dead. The table, its columns (`id`, `student_id`, `habit_type`, `date`, `completed_at`, `created_at`) and its 7 RLS policies all exist and are correct — but nothing populates it during normal use.

Aspect	Finding
Live row count	<code>habit_logs</code> = 0 rows. The parallel table <code>habit_tracking</code> = 3,010 rows but all are seed data (max date 2026-05-19, 70 students).
Only writer	<code>useSessionCompletion.ts:271</code> (study-session completion) — and it writes via <code>from("habit_logs" as never)</code> with a comment "table may not exist." No login / submit / journal / read flow writes <code>habit_logs</code> . In production it has produced zero rows.
Readers that silently get nothing	<code>useTodayView.ts:218</code> (the student Today-view habit chips) plus edge functions <code>process-streak</code> (comeback check), <code>perfect-day-prompt</code> , <code>check-badges</code> (perfect_week badge), <code>compute-habit-correlations</code> , <code>update-challenge-progress</code> (habit-streak), and <code>export-student-data</code> . All read an empty table.
User-visible impact	Today-view habit chips never tick; the perfect_week badge can never be earned; habit-correlation insights are empty; challenge habit-streaks read zero; GDPR export omits habit data.
Root cause	The daily-habit feature was built twice on two tables (<code>habit_logs</code> narrow rows vs <code>habit_tracking</code> wide booleans), and the real login/submit/journal/read flows write to neither . This is the upstream cause shared with S-1 (streak) and S-2 (perfect day).
Fix	Remediation R-7: pick ONE habit table and have the actual login/submit/journal/read flows write to it; that single change unblocks the Today view, the heatmap, the streak driver, the perfect-day payout, and the perfect_week badge.

7 • Parent role

Feature	Stack path	Status	Notes
Dashboard	ParentDashboard.tsx; useParentKPIs	NO-DATA (partial)	Only linkedChildren is real; other KPIs hardcoded 0
Linked-students view	useLinkedChildren:24-92	WORKING	verified=true enforced client + RLS; avg_attainment hardcoded 0
My Children	ParentChildrenPage	NO-DATA (partial)	XP/streak/level OK; courses count always 0
Child Progress (attainment)	ParentProgressPage.tsx:51-100	BROKEN	Reads student_courses first → no parent RLS → empty
Child Attendance	ParentAttendancePage.tsx:33-160	BROKEN	Same student_courses-first pattern → empty
Parent Planner (read-only)	ParentPlannerView.tsx	WORKING (partial)	Planner tables have parent policies; deadlines empty (courses gap)
Parent Profile page	ParentProfilePage.tsx	ORPHAN	No route imports it; route uses shared ProfilePage

Ranked issues

BLOCKER **P-1 • Progress & Attendance pages empty for every parent.** Both queries start from `student_courses`, which has **no parent SELECT policy** (live policy count = 0). **Live test refinement (Pass 3):** a real parent session can in fact read the downstream data — `outcome_attainment` (21 rows), `attendance_records` (90), `courses` (3) are all visible — but the page query short-circuits on `if (!enrollments || enrollments.length === 0) return []` (`ParentProgressPage.tsx:52-55`) because the `student_courses` read returns 0. So the page is empty even though the data is reachable. **This is a correctness/UX break, not a data leak.** Fix has two valid options: add a parent SELECT policy on `student_courses` (+ `courses`, `course_sections`, `class_sessions`, all confirmed missing parent policies), OR rewrite the query to derive enrolled courses from the already-visible `outcome_attainment`. Evidence: `20260428000003_optimize_rls_policies.sql:96-112`; `ParentProgressPage.tsx:60-64`; `ParentAttendancePage.tsx:64-68`.

P-2 • Dashboard KPIs are placeholder zeros — `useParentKPIs` `hardcodes` `totalCourses/avgAttainment/upcomingDeadlines = 0`; `useLinkedChildren` `hardcodes` `avg_attainment = 0` `and` `enrolled_courses` `is 0` due to the same

P-5 • orphan ParentProfilePage; P-6 • no parent "Profile" nav entry though the route exists.

What the RLS boundary gets right. Where parent policies exist, they are correctly scoped to *verified linked children* (`parent_student_links ... verified = true`): `outcome_attainment`, `student_gamification`, `attendance_records`, `study_sessions/planner_tasks/weekly_goals`, `evidence/submissions`, `profiles` (via a SECURITY DEFINER helper). A parent cannot see an unlinked or unverified child. The bug is missing policies on the *enrollment* tables, not a leak.

8 · Cross-cutting infrastructure

Area	Stack path	Status	Notes
Centralised realtime manager	useRealtime.ts:46-170	WORKING	Channel dedup, exponential backoff, polling fallback, unmount cleanup, scoped filters
Notifications read	useNotifications.ts:42-59	WORKING	Correct columns; RLS scoped to user_id
Notification realtime (live bell)	useNotificationRealtime.ts:33-67	NOT-WIRED	Hook correct but never mounted in production
Sentry consent gating	App.tsx:26 vs main.tsx:27	BROKEN	initSentry() runs unconditionally → consent bypassed
RouteGuard role access	RouteGuard.tsx:21-41	WORKING	Client guard + RLS enforcement
CORS headers (2 edge fns)	score-reflection-quality, generate-reflection-digest :12-13	BROKEN	x-content-type should be x-client-info
i18n / RTL	ParentPlannerView.tsx:118-205	PARTIAL	Hardcoded English in planner view

Ranked issues

- BLOCKER** I-1 · Sentry initialises before consent. App.tsx:26 calls initSentry() at module load, gated only on DSN presence — not on consent. main.tsx:27 separately calls the consent-gated initAnalyticsIfConsented() , but because the unconditional init already ran, the consent path's isInitialized() guard makes it a no-op. The two paths also differ (App.tsx enables Session Replay; consent path does not). For a FERPA/GDPR-sensitive education product targeting Qatar, error/replay capture starting without consent is a privacy-compliance issue, not just a bug.
- I-2 · CORS header typo breaks a browser-invoked function. score-reflection-quality/index.ts:12-13 advertises x-content-type instead of x-client-info ; supabase.js always sends x-client-info , so a strict preflight can reject.
- I-3 · Notification realtime never mounted. useNotificationRealtime is correctly built but has zero production callers (only a test). NotificationBell / GlobalHeader don't invoke it, so the bell badge only updates on refetch/remount.
- I-4 · ParentPlannerView not localised. I-5 · orphan edge fn generate-reflection-digest and orphan page ParentProfilePage .

9 · Retrospective — the badge-XP saga (the questions you asked)

You asked, directly: *"why is this causing the issue, why didn't we work to resolve this, and why did you use the bad approach earlier?"* Here is the honest engineering account of the badge/XP edge-function work that immediately preceded this audit.

Q1 · What was actually broken?

Two distinct defects stacked on top of each other:

- **The original blocker (B-1):** `check-badges` wrote earned individual badges to a `student_badges` table while the UI read a different `badges` table, and `award-xp` never triggered `check-badges` after an XP award. So badges could never appear, and the badge-check rarely ran. This is why production showed `badges = 0` for 71 students with 4,119 XP transactions.
- **The hidden second defect (discovered during the fix):** once badges *were* being awarded, the follow-on "award the badge's XP" call (and the `award-xp` → `check-badges` trigger) failed with a gateway 401. No server-to-server `award-xp` call had ever succeeded — visible in the ledger as zero rows for `badge`, `tutor_engagement`, `challenge_reward` and `improvement_bonus` sources.

Q2 · Why was the second defect happening? (root cause)

The functions were deployed with the platform setting `verify_jwt = true`, which makes Supabase's API gateway validate the `Authorization: Bearer` token *as a JWT* before the request ever reaches the function code. This project's injected `SUPABASE_ANON_KEY` and `SUPABASE_SERVICE_ROLE_KEY` are the **modern non-JWT sb_ format** (confirmed: the publishable key is `sb_publishable_`). A non-JWT string can never pass a JWT-verification gate. So every function-to-function call — which sends one of those keys as the bearer — was rejected at the gateway with 401 *before* the function's own (correct) in-handler auth logic could run. The function code was fine; the platform gate in front of it was incompatible with the key format the platform itself injects.

Q3 · Why wasn't this caught earlier?

- **It was latent behind the first blocker.** Because badges were never awarded end-to-end (B-1), the badge→XP sub-call was never actually exercised in production. The 401 only became reachable *after* B-1 was fixed. Fixing one bug exposed the next — classic layered failure.
- **The ledger evidence was never read this way before.** The "zero rows for every server-to-server XP source" signal was sitting in the database the whole time, but no prior check queried the source distribution. Pass 2 of this audit is exactly that query.
- **Edge functions don't fail loudly.** The calls were fire-and-forget with `.catch(console.error)`; a 401 logged to a function-log nobody was tailing. There was no test asserting "a submission produces a `badge` XP row."

Q4 · Why did the earlier approach fail — twice — before the right fix?

This is the part worth being candid about. The fix went through three bearer-token attempts before the real cause was identified:

1. **Attempt 1 — send the service-role key as the bearer.** Reasoning: "service role should be all-powerful." Result: gateway 401. *Why it was wrong:* the injected service-role key is the non-JWT `sb_secret_` format, so `verify_jwt` rejects it.
2. **Attempt 2 — send the anon JWT as the bearer + service-role in a header.** Reasoning: "the anon key is a JWT, so the gate will pass, and the handler can read the secret from a custom header." Result: still 401. *Why it was wrong:* on this project the anon key is *also* the modern non-JWT format, so it failed the same gate.
3. **Attempt 3 (correct) — deploy both functions with `verify_jwt = false`** and rely on the already-robust in-handler auth (service-role exact-match via `x-internal-auth`, plus user-JWT ownership checks for student calls). Result: 200; badges and badge-XP both land; verified live with two real students.

The honest mistake: the first two attempts kept changing *which token* was sent, treating the symptom (a 401) as an auth-token problem. The actual variable was *where* auth was being enforced (gateway vs handler) and *what format* the platform keys were. The diagnostic that should have come first — "does the gateway return 401 even for a known-valid token, i.e. is this a gateway gate or a handler rejection?" — was run third, not first. Once that diagnostic matrix was run (legacy JWT → in-handler 403 = gateway accepts; injected key → 401 = gateway rejects), the root cause was obvious and the fix was immediate.

The general lesson (worth a steering rule). When a 401/403 appears on a server-to-server call, *first* determine whether it is the **gateway** or the **handler** talking, before changing any credential. A gateway 401 and a handler 403 look similar in logs but have opposite fixes. Changing tokens without that distinction is guessing. The repo's own guidance — "if an approach has failed twice, diagnose the root cause rather than making incremental patches" — is exactly the rule that was violated on attempts 1–2 and honoured on attempt 3.

Q5 · Is the fix safe? Did turning off `verify_jwt` weaken security?

No meaningful weakening. `verify_jwt` at the gateway is a coarse "is this any valid JWT" check. Both functions enforce their own, stricter auth *inside* the handler: `award-xp` and `check-badges` accept a service-role secret only via the `x-internal-auth` header (exact-match) for server-to-server calls, and for end-user calls they validate the user's JWT with `getUser()` and enforce ownership (a student can only act on their own id — verified live: a cross-user call returns 403). Turning off the redundant gateway gate removed the *only* thing blocking legitimate internal calls while leaving the real authorization intact. The change is recorded in `scripts/deploy-edge-functions.sh` (`NO_VERIFY_JWT_FUNCTIONS`) so a future batch deploy doesn't silently re-enable the gate and re-break it.

Q6 · Current state of that work

The badge fix and the `verify_jwt=false` redeploy are **live and verified** (56 individual badges, 2 badge-XP transactions — the first ever — and a confirmed no-infinite-loop). However, the edited source for `check-badges`, `award-xp` and `deploy-edge-functions.sh` is currently **uncommitted on the local main branch** and has not yet gone through a PR. **Action required:** move these to a feature branch and open a PR so production code matches what is deployed (and so CI/property tests run against it). This is item R-0 in the plan below.

10 · Prioritised remediation plan

Ordered by risk-adjusted priority. "Effort" is rough engineer-days. None require a schema redesign.

ID	Fix	Sev	Effort	Approach (summary)
R-0	Commit the deployed badge/verify_jwt fix to a PR	BLOCKER	0.25	Branch + PR the 3 modified files; run lint/tsc/test; this only reconciles repo with prod
R-1a	Shared edge-fn role gate (403 for all coordinators/admins/teachers)	BLOCKER	0.5	generate-course-file, generate-accreditation-report, ai-feedback-draft (and any fn using the pattern) read role from JWT <code>app_metadata</code> (empty). Resolve role/institution from <code>profiles</code> by <code>user.id</code> instead. One shared helper fixes all
R-1b	N-1 · Planner task status mismatch	BLOCKER	0.25	usePlannerTasks writes <code>pending/completed</code> ; DB CHECK allows <code>todo/in_progress/done/deferred</code> . Align hook to DB values (and remove the <code>as never</code> that hid it). Create + complete both broken today
R-1c	N-2 · Quiz gen <code>courses.institution_id</code> drift — ✓ FIXED & DEPLOYED	BLOCKER	done	Now joins <code>program_id+programs</code> ; reads TUTOR_PRIMARY_MODEL; conditional response_format. Verified 200 with real Kimi questions
R-1d	N-3 · Client XP sources rejected 403	BLOCKER	0.5	study_session/wellness/planner_task/weekly_goal/review XP all 403 because hooks call award-xp as the student. Either add these to award-xp self-allow-list with server-enforced amounts, or route them through a trusted server path. Today they award nothing
R-1e	N-4 · AI keys + tutor OpenAI dependency — ✓ PARTIALLY FIXED	HIGH	partial	OpenRouter/Kimi key set; tutor embeddings made optional & deployed (tutor + quiz now work on Kimi). REMAINING: configure an embeddings provider for RAG citations (<code>embed-course-material</code> still OpenAI-only)
R-1f	N-5 · generate-plan-update NOT NULL bug	MED	0.1	Validate/default <code>recent_interaction_count</code> (← <code>interaction_count</code>) in generate-plan-update; it 500s on save when the caller omits it
R-1	generate-accreditation-report column drift	BLOCKER	0.5	<code>score_percent+attainment_percent</code> ; map PLO/ILO via <code>outcome_id</code> joins not <code>scope</code> ; <code>graduate_attributes.name/description</code> ; add <code>if(error) throw</code>
R-2	generate-course-file column drift (4 queries)	BLOCKER	0.75	<code>outcome_mappings</code> source/target; read scores from <code>grades</code> ; attainment_percent + scope <code>student_course/course</code> ; CQI <code>action_description/program_id</code> ; add error guards
R-3	Sentry consent gate	BLOCKER	0.25	Remove unconditional <code>initSentry()</code> in App.tsx; route solely through the consent-gated path; unify config
R-4	Parent RLS on student_courses (+courses/sections/sessions)	BLOCKER	0.5	Add parent SELECT policies scoped to verified linked children (mirror existing attainment policy); migration + replay check
R-5	Student streak driver on login	HIGH	1.0	Either invoke per-student <code>process-streak</code> on login, or add an "all active students" batch branch + point the midnight cron at it; write <code>habit_tracking.login</code> ; add integration test "login advances streak"
R-6	Perfect Day 50 XP payout	HIGH	0.5	On 4th-habit completion award idempotent <code>perfect_day</code> (key <code>perfect_day:{student}:{date}</code>) + fire check-badges('habit_log')
R-7	Record academic habit completions	HIGH	1.0	Pick ONE habit table; write login/submit/journal on their real flows; this unblocks heatmap, streak, perfect-day
R-8	Grade + journal XP awards on client	HIGH	0.5	Invoke award-xp('grade')/('journal') from useCreateGrade/useCreateJournalEntry (idempotent ref)
R-9	Grading-queue linner join filter	HIGH	0.25	Change embedded <code>assignments</code> filter to <code>assignments!linner</code> in useSubmissions
R-10	CORS x-client-info typo (2 fns)	HIGH	0.1	Replace <code>x-content-type</code> → <code>x-client-info</code> ; redeploy score-reflection-quality
R-11	Mount notification realtime hook	HIGH	0.1	Call <code>useNotificationRealtime()</code> in NotificationBell/GlobalHeader
R-12	Grade-trigger level recompute	MED	0.5	Recompute <code>level</code> + apply multipliers in trigger, or route grade XP through award-xp; add SUM reconciliation
R-13	Parent dashboard real KPIs + scope filter	MED	0.5	After R-4, compute totalCourses/avgAttainment/enrolled; add scope='individual' to badge reads
R-14	Bonus-event audit allow-list, .single() misuse, placeholders	MED	0.5	Fix auditLogger column names; ,maybeSingle() on read-by-filter; real curriculum-matrix attainment
R-15	Hygiene: remove <code>as never</code> / <code>as any</code> , orphan fns/pages, i18n planner	LOW	1.5	Drop stale casts now that types exist; delete/route orphans; localise ParentPlannerView

Suggested sequencing

- Sprint A (release-gate, ~2.5 days):** R-0 → R-4. These are the four BLOCKERS plus the repo/prod reconciliation. After this the accreditation artefacts are correct, privacy consent is honoured, and parents can see their children.
- Sprint B (engagement loop, ~3.5 days):** R-5 → R-11. Make the student daily loop actually function and fix the teacher grading filter + realtime bell. Do R-7 first — it unblocks R-5 and R-6.
- Sprint C (hardening, ~2 days):** R-12 → R-15. Correctness polish, audit fidelity, and type-safety/hygiene debt.

Two process changes that would have prevented most of this

- Error guards in every edge-function query.** The two PDF BLOCKERS and the badge-XP 401 all shipped silently because failures were swallowed. A lint rule or review checklist requiring `if (error) throw` (or explicit handling) on every Supabase call in `supabase/functions/**` would have surfaced all three at deploy time.
- One end-to-end "golden path" test per role against a seeded DB.** A single test that logs in as a student, submits, and asserts an XP row *and* a badge row appear would have caught B-1, the 401, S-1, S-4 and S-5 at once. The same for "generate a course file and assert non-empty sections" catches R-1/R-2.

11 · Verification & cross-reference appendix (senior-dev / QA / DevOps)

Every issue below was re-checked against **live code as it exists now** and the **live production schema/secrets** — not against seed data or a prior report. Each row states the exact verification source so you can trust it before investing in a fix. The "blast radius / don't-break" column is the cross-reference map: what else touches the same code path, so fixing one thing doesn't silently break another.

ID	Issue	Verified TRUE by	Seed-data risk?	Blast radius / don't-break-these
A-0 / C-0	PDF + AI role gate reads role from JWT (empty) → 403 for all staff	Live JWT decode: <code>app_metadata={provider,providers}</code> , no role. Live: no custom-access-token hook exists (only <code>auth.jwt</code>). Source grep: 6 functions use <code>app_metadata?.role</code>	NO — independent of data	Fix all 6 together: <code>_shared/auth.ts</code> , send-email-notification, generate-course-file, generate-accreditation-report, check-login-rate, calculate-attainment-rollup. (ai-feedback-draft + ai-module-suggestion already fixed.) Best fix = add a custom_access_token_hook that injects role+institution_id from <code>profiles</code> once, then all functions work — but verify each function's fallback still allows service-role calls (cron/edge-to-edge) before removing the metadata read.
A-1	generate-accreditation-report column drift	Live schema: <code>outcome_attainment</code> has <code>attainment_percent</code> not <code>score_percent</code> ; enum has no PLO/ILO; <code>graduate_attributes</code> = name/description not title/code. Source confirms the bad selects	NO	Same <code>outcome_attainment / graduate_attributes</code> columns are read correctly by <code>useAdminDashboard / useDepartmentAnalytics</code> — copy their working query shape. Don't change the tables; only the function's SELECTs.
C-1	generate-course-file 4 broken queries	Live schema: <code>outcome_mappings</code> =source/target (not child/parent); <code>submissions</code> has no <code>score_percent</code> ; <code>cqi_action_plans</code> =action_description/program_id (not title/course_id). All confirmed	NO	Scores live on <code>grades.score_percent</code> (joined via submission). The coordinator app hooks (<code>useCQIPPlans / usePLOWMappings</code>) already use the correct columns — mirror them. Adding <code>if(error) throw</code> here will surface any future drift.
N-1	Planner task create/complete violates CHECK	Live CHECK: <code>todo/in_progress/done/deferred</code> . Source: <code>usePlannerTasks.ts:59</code> writes <code>"pending"</code> , <code>:197</code> writes <code>"completed"</code>	NO — pure code/constraint mismatch	Two valid fixes: (a) change the hook to DB values, OR (b) widen the CHECK. Prefer (a). BUT: the row→domain mapper (<code>mapTask</code> defaults <code>status</code> to <code>"pending"</code>) and <code>src/types/planner.ts PlannerTask["status"]</code> union must be aligned in the SAME change, plus the Today-view/planner UI badges that switch on status — grep <code>PlannerTask["status"]</code> before merging.
N-2	Quiz gen 404 on courses.institution_id	Live schema: <code>courses</code> has no <code>institution_id</code> (cols: program_id, teacher_id, ...). FIXED & deployed (join via programs) — re-verified 200 with real Kimi questions	NO	Already fixed. The join shape <code>programs(institution_id)</code> is the canonical way institution is derived everywhere — safe.
N-3	study/wellness/planner/goal/review XP all 403	Live: award-xp self-allowlist = <code>["login","submission","journal"]</code> . Source: 4 hooks / 6 sources invoke award-xp as the student → 403 (swallowed). Confirmed by live 403	NO	Sources to unblock: <code>planner_task</code> , <code>study_session</code> , <code>review_session</code> , <code>review_cycle_complete</code> , <code>weekly_goal</code> , <code>wellness_habit</code> . Don't just add them to the self-allowlist blindly — award-xp already server-enforces fixed amounts per source (study_session cap 60, planner 10, wellness from institution_settings). Add them to <code>selfTriggeredSources</code> AND confirm each has a server-enforced amount branch so a student can't pass an arbitrary <code>xp_amount</code> . The XP ledger/level math downstream is healthy (verified) and won't break.
N-5	generate-plan-update NOT NULL interaction_count	Live schema: <code>interaction_count integer NOT NULL, no default</code> . Source: insert sets it from request <code>recent_interaction_count</code> which is unvalidated → NULL when omitted	NO	Validate/default the param. Caller is <code>chat-with-tutor</code> (now working on Kimi) — when you wire the real caller, ensure it passes the count. Low blast radius.
P-1	Parent Progress/Attendance empty	Live: parent SELECT policy count on <code>student_courses</code> = 0 ; also missing on courses/course_sections/class_sessions. Source: both pages query <code>student_courses</code> first and <code>return []</code> when empty	NO — policy genuinely absent	Two valid fixes: (a) add parent RLS policies (mirror the existing verified-link policy on <code>outcome_attainment</code>), OR (b) rewrite pages to derive courses from already-visible <code>outcome_attainment</code> . If you choose (a), the policy MUST use the same <code>parent_student_links ... verified=true</code> predicate as the other parent policies — do NOT broaden it, or you create a cross-student leak. Re-run the parent isolation probe after.
S-1	Login never advances streak	Live: process-streak WORKS when called per-student (verified streak→1). Source: <code>AuthProvider.signIn</code> only logs activity; no process-streak/award-xp('login') call; midnight cron passes no student_id	PARTIAL — seed streaks exist (0-31) but are static; no runtime advance	Fix = invoke process-streak on login OR add batch mode to the cron. process-streak also writes streak milestones — check-badges ; confirm the streak_7/14/... badges still fire after the change (that chain is verified working). Don't double-fire if you add both login + cron paths (it's idempotent per day, but verify).
S-5	Journal +20 XP advertised, never awarded	Live: <code>journal</code> IS in the self-allowlist (so not blocked). Source: <code>useCreateJournalEntry</code> has zero award-xp call; UI promises "Earn 20 XP per entry" (StudentJournalPage:128,254)	NO	Unlike N-3, journal is allowlisted — just add the award-xp('journal') call on successful create (≥50 words, per the UI rule). The 50-word gate logic exists in the UI; mirror it server-side or pass it. award-xp handles the idempotent reference_id.
I-1	Sentry initialises before consent	Source: <code>App.tsx:26</code> calls <code>initSentry()</code> unconditionally; <code>main.tsx:27</code> has the consent-gated path that becomes a no-op	NO	Remove the unconditional init. Don't remove the consent-gated path. Verify error reporting still works AFTER consent is granted (the <code>isInitialized()</code> guard means order matters).
I-2	CORS x-content-type typo	Source: <code>score-reflection-quality + generate-reflection-digest</code> :12-13 send <code>x-content-type</code> (should be x-client-info)	NO	Trivial. score-reflection-quality is browser-invoked (real impact); the digest fn is orphaned.
AI tutor	Tutor + quiz + role-gated AI	FIXED & deployed, re-verified live: real Kimi chat, integrity guard, quiz 200. <code>tutor_llm_logs</code> = 5 successes	NO	RAG citations still need an embeddings provider. The 4 AI fixes are deployed but NOT committed to git (R-0 applies).

Two findings that are NOT pure bugs — read before "fixing": (1) `habit_logs empty` is a *consequence* of S-1/S-3 (nothing writes habits at runtime), not an independent bug — fixing the habit-write path fixes it. (2) **The xp_total "drift" originally reported is NOT occurring live** (0 students drift); the real residual risk is the grade DB-trigger not recomputing `level` . Don't "fix" a drift that isn't happening — just align the trigger's level recompute.

The single highest-leverage fix. Add a Supabase `custom_access_token_hook` that injects `role` and `institution_id` (from `profiles`) into the JWT. That one change makes A-0, C-0, and the 6-function role-gate class all correct at once, AND lets every edge function trust the token instead of an extra DB round-trip. Sequence it carefully: deploy the hook first, confirm new logins carry the claims (decode a fresh JWT), THEN the functions' existing `app_metadata?.role` reads start working automatically — you may not even need to touch the 6 functions. Verify service-role/cron paths (which have no user JWT) still hit their `isServiceRole` branch.

How each finding was verified (audit integrity)

- Schema facts** (column names, CHECK constraints, enums, RLS policy counts, NOT NULL): read from `information_schema / pg_constraint / pg_policies` on the live production DB at audit time — not from migrations or types.
- Behaviour** (403/404/500/200, XP awarded or not, tutor answer): executed as real authenticated demo users against production edge functions; responses quoted verbatim.
- Code claims** (which hook calls what): every one cites `file:line` from current source, re-read during this verification pass.
- Seed vs runtime:** where data existed, I checked timestamps/source to distinguish seeded rows from runtime-generated ones (e.g. all `login / perfect_day` XP rows carry mid-May seed timestamps; no runtime XP of those sources exists).

End of audit. Produced across five verification passes (static trace → live DB reads → live per-role functional test → seed-and-test of empty domains → live AI test with fixes). Findings re-verified against live code and live schema in a dedicated cross-reference pass (§11). Edge-function fixes for badges, AI role-gates, quiz, and tutor embeddings were deployed to production during the audit and are flagged for git reconciliation (R-0).